

Cell 1: Imports & Environment Setup

```
In [18]: import os
import glob
import cv2
import numpy as np
import pandas as pd
import torch
from tqdm import tqdm
from pathlib import Path
from sahi import AutoDetectionModel
from sahi.predict import get_sliced_prediction
import warnings
warnings.filterwarnings('ignore')

# Device selection (MPS for Apple Silicon, fallback to CPU)
device = "mps" if torch.backends.mps.is_available() else "cpu"
print(f"✅ Using device: {device}")
```

✅ Using device: mps

Cell 2: Configuration & Model Paths

```
In [19]: BASE_DIR = "/Users/macbook/Documents/ITC8/Internship/AI Farm/Testing/car_def
TEST_IMG_DIR = os.path.join(BASE_DIR, "data/processed/yolo_seg/images/test")
TEST_DATA_YAML = os.path.join(BASE_DIR, "data/processed/yolo_seg/data.yaml")

# Model paths (Update the mlruns hashes to your actual best.pt locations)
MODELS = {
    "surgical_early": os.path.join(BASE_DIR, "runs/segment/seesaw_surgical_e
    "baseline_m5": os.path.join(BASE_DIR, "runs/segment/stage1_head_warmup_7
    "model_4": os.path.join(BASE_DIR, "runs/segment/model5_resume_adapt_7cls
    "objectness_branch": os.path.join(BASE_DIR, "runs/segment/seesaw_sugical
}

# Load Eval Dataset Class Order (The "Ground Truth" ordering)
with open(TEST_DATA_YAML) as f:
    eval_cfg = yaml.safe_load(f)
eval_names = eval_cfg['names']
eval_name_to_idx = {v: int(k) for k, v in eval_names.items()}
NUM_CLASSES = len(eval_names)

# Gather Test Images
exts = ["*.jpg", "*.jpeg", "*.png", "*.JPG", "*.PNG"]
image_paths = []
for ext in exts:
    image_paths.extend(glob.glob(os.path.join(TEST_IMG_DIR, ext)))
image_paths = sorted(list(set(image_paths)))
```

```
print(f"✅ Eval Dataset Order: {eval_names}")
print(f"✅ Found {len(image_paths)} test images.")
```

✅ Eval Dataset Order: {0: 'broken_lamp', 1: 'corrosion', 2: 'crack', 3: 'dent', 4: 'disjoint_part', 5: 'glass_shatter', 6: 'scratch'}

✅ Found 593 test images.

Cell 3: Memory-Optimized Helpers (Mask-IoS & mAP50)

```
In [20]: def load_yolo_masks(label_path, img_h, img_w):
    """Loads YOLO seg labels and crops masks to their bounding boxes to save
    masks = []
    if not os.path.exists(label_path): return masks
    with open(label_path, 'r') as f:
        for line in f:
            parts = line.strip().split()
            if len(parts) < 7: continue
            cls_id = int(parts[0])
            coords = list(map(float, parts[1:]))
            pts = np.array([[int(coords[i] * img_w), int(coords[i+1] * img_h)
            x1, y1 = pts.min(axis=0); x2, y2 = pts.max(axis=0)
            h, w = y2 - y1 + 1, x2 - x1 + 1
            if h <= 0 or w <= 0: continue
            mask = np.zeros((h, w), dtype=bool)
            local_pts = pts.copy(); local_pts[:, 0] -= x1; local_pts[:, 1] -= y1
            cv2.fillPoly(mask, [local_pts], 1)
            masks.append({'class': cls_id, 'mask': mask, 'bbox': (x1, y1, x2, y2)})
    return masks

def compute_map50(all_preds, all_gts, num_classes, eval_names, ios_thresh=0.5):
    """Computes per-class AP50 using Mask-IoS."""
    aps = {}
    for c in range(num_classes):
        preds_c = [p for p in all_preds if p['class'] == c]
        gts_c = [g for g in all_gts if g['class'] == c]
        preds_c.sort(key=lambda x: x['score'], reverse=True)
        tp, fp = np.zeros(len(preds_c)), np.zeros(len(preds_c))
        matched_gts = set()

        for i, p in enumerate(preds_c):
            img_idx, p_mask, p_area = p['img_idx'], p['mask'], p['area']
            px1, py1, px2, py2 = p['bbox']
            img_gts = [g for g in gts_c if g['img_idx'] == img_idx]
            best_ios, best_gt_id = 0, -1

            for j, g in enumerate(img_gts):
                g_mask, g_area = g['mask'], g['area']
                gx1, gy1, gx2, gy2 = g['bbox']
                ix1, iy1 = max(px1, gx1), max(py1, gy1)
                ix2, iy2 = min(px2, gx2), min(py2, gy2)
                if ix1 < ix2 and iy1 < iy2:
                    p_crop = p_mask[iy1-py1:iy2-py1, ix1-px1:ix2-px1]
                    g_crop = g_mask[iy1-gy1:iy2-gy1, ix1-gx1:ix2-gx1]
```

```

        inter = np.sum(p_crop & g_crop)
        smaller_area = min(p_area, g_area)
        if smaller_area > 0:
            ios = inter / smaller_area
            if ios > best_ios: best_ios, best_gt_id = ios, id(g)

    if best_ios >= ios_thresh:
        if best_gt_id not in matched_gts: tp[i] = 1; matched_gts.add(best_gt_id)
        else: fp[i] = 1
    else: fp[i] = 1

    cum_tp, cum_fp = np.cumsum(tp), np.cumsum(fp)
    recall = cum_tp / len(gts_c) if len(gts_c) > 0 else np.zeros_like(cum_tp)
    precision = cum_tp / (cum_tp + cum_fp)
    mrec = np.concatenate([0.0, recall, 1.0])
    mpre = np.concatenate([1.0, precision, 0.0])
    for i in range(mpre.size - 1, 0, -1): mpre[i - 1] = np.maximum(mpre[i], mpre[i + 1])
    i = np.where(mrec[1:] != mrec[:-1])[0]
    ap = np.sum((mrec[i + 1] - mrec[i]) * mpre[i + 1])
    aps[eval_names[c]] = ap
return aps

```

Cell 4: SAHI Evaluation Loop

```

In [21]: import logging
logging.getLogger("sahi").setLevel(logging.ERROR) # Suppress the low-conf warnings

def evaluate_model(model_name, model_path, image_paths, eval_name_to_idx, eval_names):
    print(f"\n🔄 Loading {model_name}...")

    # 1. Get Model's Internal Class Order & Build Translation Map
    yolo_model = YOLO(model_path)
    model_names = yolo_model.names
    idx_map = {int(k): eval_name_to_idx[v] for k, v in model_names.items()}
    print(f"    Model internal order: {model_names}")
    print(f"    Remapping to eval order: {idx_map}")

    # 2. Load SAHI Model
    sahi_model = AutoDetectionModel.from_pretrained(
        model_path=model_path, model_type="yolov8",
        device=device, confidence_threshold=0.01
    )

    all_preds, all_gts = [], []

    for img_idx, img_path in enumerate(tqdm(image_paths, desc=f"Evaluating {model_name}")):
        img = cv2.imread(img_path)
        img_h, img_w = img.shape[:2]

        # Load Ground Truth (already in eval dataset order)
        lbl_path = img_path.replace("/images/", "/labels/").rsplit(".", 1)[0] + ".txt"
        gts = load_yolo_masks(lbl_path, img_h, img_w)
        for g in gts: g['img_idx'] = img_idx; all_gts.append(g)

```

```

# SAHI Inference (PRODUCTION PARAMS: 1024x1024, 15% overlap, IOS)
result = get_sliced_prediction(
    img_path, sahi_model,
    slice_height=1024, slice_width=1024,
    overlap_height_ratio=0.15, overlap_width_ratio=0.15,
    perform_standard_pred=False,
    postprocess_type="GREEDYNMM",
    postprocess_match_metric="IOS",
    postprocess_match_threshold=0.5,
    verbose=0
)

# Extract, Remap, and Crop Predictions
for p in result.object_prediction_list:
    try:
        raw_cls_id = int(p.category.id)
        if raw_cls_id not in idx_map: continue # Skip unmapped class
        eval_cls_id = idx_map[raw_cls_id] # TRANSLATE TO EVAL INDEX

        score = p.score.value
        m = np.asarray(p.mask.bool_mask) > 0.5
        ys, xs = np.nonzero(m)
        if len(xs) == 0: continue
        px1, py1, px2, py2 = xs.min(), ys.min(), xs.max(), ys.max()
        cropped_m = m[py1:py2+1, px1:px2+1]

        all_preds.append({
            'img_idx': img_idx, 'class': eval_cls_id, 'score': score,
            'mask': cropped_m, 'bbox': (px1, py1, px2, py2), 'area':
        })
    except Exception: continue

aps = compute_map50(all_preds, all_gts, NUM_CLASSES, eval_names, ios_thr)
mAP50 = np.mean(list(aps.values()))
return aps, mAP50

```

Cell 5: Run Benchmark & Display Results

```

In [22]: results_summary = []

for model_name, model_path in MODELS.items():
    if not os.path.exists(model_path):
        print(f"⚠ Skipping {model_name}: Weights not found.")
        continue
    aps, mAP50 = evaluate_model(model_name, model_path, image_paths, eval_names)
    row = {"Model": model_name, "mAP50": mAP50}
    row.update(aps)
    results_summary.append(row)

df_results = pd.DataFrame(results_summary)
cols = ["Model", "mAP50"] + [eval_names[i] for i in range(NUM_CLASSES)]
df_results = df_results[cols]

df_formatted = df_results.copy()

```

```

for col in cols[1:]:
    df_formatted[col] = df_formatted[col].apply(lambda x: f"{x*100:.2f}%")

print("\n🏆 PRODUCTION-ACCURATE SAHI Mask-IoS mAP50 (1024x1024 slices)")
display(df_formatted)

```

🔄 Loading surgical_early...

Model internal order: {0: 'broken_lamp', 1: 'corrosion', 2: 'crack', 3: 'dent', 4: 'disjoint_part', 5: 'glass_shatter', 6: 'scratch'}

Remapping to eval order: {0: 0, 1: 1, 2: 2, 3: 3, 4: 4, 5: 5, 6: 6}

Evaluating surgical_early: 100%|██████████| 593/593 [03:57<00:00, 2.50it/s]

🔄 Loading baseline_m5...

Model internal order: {0: 'dent', 1: 'scratch', 2: 'crack', 3: 'glass_shatter', 4: 'broken_lamp', 5: 'corrosion', 6: 'disjoint_part'}

Remapping to eval order: {0: 3, 1: 6, 2: 2, 3: 5, 4: 0, 5: 1, 6: 4}

Evaluating baseline_m5: 100%|██████████| 593/593 [11:17<00:00, 1.14s/it]

🔄 Loading model_4...

Model internal order: {0: 'dent', 1: 'scratch', 2: 'crack', 3: 'glass_shatter', 4: 'broken_lamp', 5: 'corrosion', 6: 'disjoint_part'}

Remapping to eval order: {0: 3, 1: 6, 2: 2, 3: 5, 4: 0, 5: 1, 6: 4}

Evaluating model_4: 100%|██████████| 593/593 [09:44<00:00, 1.02it/s]

🔄 Loading objectness_branch...

Model internal order: {0: 'broken_lamp', 1: 'corrosion', 2: 'crack', 3: 'dent', 4: 'disjoint_part', 5: 'glass_shatter', 6: 'scratch'}

Remapping to eval order: {0: 0, 1: 1, 2: 2, 3: 3, 4: 4, 5: 5, 6: 6}

Evaluating objectness_branch: 100%|██████████| 593/593 [02:38<00:00, 3.74it/s]

🏆 PRODUCTION-ACCURATE SAHI Mask-IoS mAP50 (1024x1024 slices)

	Model	mAP50	broken_lamp	corrosion	crack	dent	disjoint_part	g
0	surgical_early	67.01%	81.61%	66.97%	48.39%	72.91%	46.37%	
1	baseline_m5	63.65%	96.03%	6.70%	63.51%	79.70%	41.20%	
2	model_4	57.93%	89.12%	9.41%	58.73%	72.64%	20.88%	
3	objectness_branch	66.56%	80.36%	68.68%	48.56%	71.59%	44.92%	

```

In [23]: from ultralytics import YOLO
import yaml

# 1. What class ORDER does Model 5 have internally?
m5 = YOLO(os.path.join(BASE_DIR, "runs/segment/stage1_head_warmup_7cls_exter
print("Model 5 names:      ", m5.names)

# 2. What class ORDER does the working model have?
se = YOLO(os.path.join(BASE_DIR, "runs/segment/seesaw_surgical_early/weights
print("surgical_early names:", se.names)

# 3. What class ORDER does the eval dataset define?
with open("/Users/macbook/Documents/ITC8/Internship/AI Farm/Testing/car_defe
    cfg = yaml.safe_load(f)
print("Eval dataset names:  ", cfg["names"])

```

```
Model 5 names:      {0: 'dent', 1: 'scratch', 2: 'crack', 3: 'glass_shatter', 4: 'broken_lamp', 5: 'corrosion', 6: 'disjoint_part'}
surgical_early names: {0: 'broken_lamp', 1: 'corrosion', 2: 'crack', 3: 'dent', 4: 'disjoint_part', 5: 'glass_shatter', 6: 'scratch'}
Eval dataset names:  {0: 'broken_lamp', 1: 'corrosion', 2: 'crack', 3: 'dent', 4: 'disjoint_part', 5: 'glass_shatter', 6: 'scratch'}
```

```
In [ ]: !jupyter nbconvert --to webpdf --allow-chromium-download "benchmark.ipynb"
```